

Scrumconix: Agile and documented method to AGSD

Luis T. Portela

Programa Educativo de Ingeniería en TI
Universidad Tecnológica del Sur de Sonora
Cd. Obregón, México

Gilberto Borrego

Facultad de Ciencias
Universidad Autónoma de Baja California
Ensenada, México

Abstract—Many companies have adopted agile software development (ASD), mainly due to it can handle scarce requirements. However, some unsolved challenges exist in ASD, particularly in global software development (GSD) companies (known as AGSD). These challenges include ASD lax documentation contrasted by the methodological standardization required in GSD, due to its inherent distances. Lax documentation leads to documentation debt and architectural knowledge (AK) vaporization, which cause negative effects on the development process and on the product itself. In order to reduce these effects, we propose Scrumconix, a hybrid method that uses a lightweight approach to document in AGSD environments, which also aims to decrease the effect of linguistic and cultural distances. In addition, we present preliminary results of Scrumconix implementation in a Mexican AGSD company.

Keywords— *agile global software development; architectural knowledge vaporization; documentation debt; Scrum; ICONIX.*

I. INTRODUCTION

Agile software development (ASD) have become a popular and effective way to deliver software in situations with scarce requirements [1]. Many companies, including those that apply global software development (GSD), have adopted ASD [2], since they have recognized its benefits (e.g. manage changing priorities, team productivity and project visibility [3]). However, unsolved challenges exist in ASD, particularly in GSD companies (named AGSD companies). One of the main challenges in AGSD relates to software documentation. On the one hand, methodological standardization is preferred in GSD, which involves comprehensive documentation [4]. This preference is driven by the inherent distances in GSD: linguistic, cultural, physical and temporal [5]. On the other hand, in ASD individuals' interactions and working software are preferred over comprehensive documentation, processes and tools [6]. This reduces the formality and the quantity of documentation [7]. Further, ASD practices dominate GSD practices in an AGSD environment [8]. Thus, there is a lack of formal documentation in AGSD, which leads to architectural knowledge (AK) vaporization and documentation debt [9].

In ASD, documentation debt is solved with teammates' interaction, however in AGSD, the four distances hinder this solution. This lack of documentation causes defects in software evolution and maintenance, lack of visibility for project monitoring and technical solutions, and poorly understood requirements [10]. Furthermore, ASD developers omit documents due to sprint pressure [9], and because they consider documentation as a secondary and non-creative activity [11]. However, ASD developers recognize that documentation is important to understand and communicate about a software

system [12]. In order to reduce AK vaporization and documentation debt in AGSD, we propose a hybrid method (called Scrumconix), which uses lightweight documentation, mainly based on diagrams, to decrease the effect of linguistic and cultural distances. Also, we present preliminary results of Scrumconix implementation in a Mexican AGSD company.

II. SOFTWARE DEVELOPMENT METHOD DEFINITION

Scrumconix uses the project managing framework of Scrum [13], and part of the software engineering guide of ICONIX [14]. It is basically composed of two parts: Sprint Zero and Sprint One to N, described below. The Scrum activities that are not described is because they are unchanged.

A. Sprint Zero

In this sprint the Scrum Team gets the overall project understanding, through the following phases.

1) *Overall requirements gathering.* The product owner gather overall project requirements supported with Graphic User Interface Storyboards (GUIs) (i.e. mockups, wireframes, etc.). Namely, while the product owner is gathering the client requirements, he could draw the corresponding GUIs, to better confirm the requirements understanding.

2) *Domain and use case modeling.* The product owner leads a two-hour meeting where, based on the product owner's project knowledge and GUIs, the Scrum team develops the domain model. This model is an UML class diagram (without attributes and methods), that represents the real-world objects involved in the project. The domain model is refined throughout the project to reflect the current problem understanding. Then, in another session(s), the Scrum team develops the use cases diagram, supported by the domain model, GUIs and the product owner's project knowledge. Subordinated use cases [15] are preferred since they are more useful in the estimation phase.

3) *Reference software architecture.* Based on the use cases diagram, domain model and product owner's project knowledge about non-functional requirements, the Scrum team has the elements to propose a reference architecture (RA) [16] to the project.

4) *Overall project estimation.* Once the Scrum team knows the involved use cases and the project RA, the use case points technique could be applied for estimation. In this manner, a project's end date could be obtained, as well as, project effort in terms of hour/person.

5) *GUIs refinement*. The product owner decides which use cases will be developed in the next sprint, and he gathers details of the involved GUIs, to have enough elements to plan the next sprint. This is because the requirements gathering phase doesn't delve too deeply in details.

6) *Sprint planning*. In a four-hour meeting, the product owner and Scrum team plan the next sprint. In this meeting, the product owner presents the chosen use cases and their detailed GUIs. Since use cases were just identified, product owner and scrum team describe them in a collaborative manner using robustness diagrams [14]. In ICONIX, this diagram is used to analyze use cases description and to identify use cases objects. These objects could be boundary (GUIs or another system interfaces), entity (domain entities), and controllers (software actions). In our experience, we have observed that use case text could be omitted, due to robustness diagram (along with UML notes) could be clear enough to describe use cases flows. Thus, a robustness diagram is a graphic version of a use case textual description. While Scrum team draws the robustness diagrams, the domain model could be updated with class attributes and even new entities. Further, each object in the robustness diagram represents a sprint task, for instance: a boundary object could be a GUI coding task, and a control object could be a method coding task. Each of these tasks are small enough to be estimated through any agile estimation technique. Thus, new benefits could be obtained using robustness diagrams in sprint planning: (1) a lightweight documentation for each use case, (2) a collaborative way to get agreement about detailed functionality of each use case, and (3) easy task identification and estimation.

In summary, with a few artifacts (GUIs, domain model, use cases diagram, reference architecture, and robustness diagrams), the understanding of a software project could be obtained. Thus, Sprint Zero is aligned with agile manifesto [6].

B. Sprint One to N

The subsequent sprints (one to N) are composed of coding, testing, and refinement of: GUIs, use cases, and domain model. We describe Sprint One to N in the next paragraphs.

1) *Coding*. The Scrum team develops the corresponding use cases supported with robustness diagrams, RA, the domain model and the use case diagram.

2) *GUI Storyboards refinement*. It is conducted in the same way as in the previous sprint. The Product owner could receive new requirements.

3) *Test case development*. Some AGSD teams have a dedicated testing team. In this case, when the Scrum team is coding, the testing team develops test cases based on robustness diagrams and GUIs.

4) *Testing/Bug fixing*. When a testing team exists, they conduct the test cases execution. When the same Scrum team tests their own code, we recommend using the corresponding robustness diagrams to follow all the use cases flows.

5) *Sprint planning*. It conducted in the same way as in the previous sprint. The RA could be refined due to new requirements or the discovery of new details.

III. PRELIMINARY RESULTS AND CONCLUSIONS

Scrumconix was implemented with two teams (9 members total) in a Mexican AGSD company. After two months since its implementation, we obtained participants' opinions through a questionnaire and a focus group. They felt that the domain model domain model was useful to visualize the project's scope, use cases diagrams helped them identify software functionality, and robustness diagrams provided a clear and common language. Also, these artifacts were sufficient to perform an accurate estimation and to produce test cases. Participants also said that the use of diagrams improved the communication between local and foreign team members. Finally, we can conclude that Scrumconix was perceived as useful and lightweight to document and understand a software project. However, when the client requests a change over a developed use case, participants do not update the corresponding artifacts. Also, the participants did not report any experiences about the use case points technique because they did not apply it. These issues have to be addressed in later studies.

REFERENCES

- [1] J. Cho, "A Hybrid Software Development Method For Large-Scale Projects: Rational Unified Process With Scrum," *J. Issues Inf. Syst.*, 2009.
- [2] VersionOne, "10th annual state of agile report," 2015.
- [3] A. Smith, G., Sidky, *Becoming Agile: ...in an imperfect world*. Manning, 2009.
- [4] D. Šmite, P. J. Ågerfalk, and N. B. Moe, "Fundamentals of Agile Distributed Software Development," in *Agility Across Time and Space: Implementing Agile Methods in Global Software Projects*, 2010, pp. 1–341.
- [5] A. Vizcaino, F. Garcia, M. Piattini, and P. Garrido, "El desarrollo global del software," in *Desarrollo Global de Software*, A. Vizcaino, F. Garcia, and M. Piattini, Eds. Ra-ma, 2014, pp. 37–58.
- [6] M. Beck, K. Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, "The Agile Manifesto." 2001.
- [7] D. Turk, R. France, and B. Rumpe, "Limitations of agile software processes," in *Third International Conference on Extreme Programming and Flexible Processes in Software Engineering*, 2002, pp. 43–46.
- [8] V. Clerc and P. Lago, "Architectural Knowledge Management Practices in Agile Global Software Development," 2011.
- [9] H. M. Sneed, "Dealing with Technical Debt in agile development projects," *Lect. Notes Bus. Inf. Process.*, vol. 166 LNBP, pp. 48–62, 2014.
- [10] N. Uikey, U. Suman, and A. Ramani, "A Documented Approach in Agile Software Development," *Int. J. Softw. ...*, vol. 2, no. 2, pp. 13–22, 2011.
- [11] T. Clear, "Documentation and Agile Methods: Striking a Balance," *SIGCSE Bull.*, vol. 35, no. 2, pp. 12–13, Jun. 2003.
- [12] Y. Shmerlin, I. Hadar, D. Kliger, and H. Makabee, "To Document or Not to Document? An Exploratory Study on Developers' Motivation to Document Code," in *Advanced Information Systems Engineering Workshops: CAiSE 2015 International Workshops*, A. Persson and J. Stirna, Eds. Stockholm, Sweden: Springer International Publishing, 2015, pp. 100–106.
- [13] K. Schwaber and J. Sutherland, "The scrum guide," *Scrum. org*, Oct., vol. 2, p. 17, 2011.
- [14] D. Rosenberg and M. Stephens, *Use Case Driven Object Modeling with UML: Theory and Practice*. Apress, 2008.
- [15] A. Cockburn, "Structuring Use Cases with Goals," *J. Object-Oriented Program.*, vol. Sept-Oct a, 1997.
- [16] P. Clements, F. Bachmann, L. Bass, D. Garlan, J. Ivers, R. Little, R. Nord, and J. Stafford, *Documenting Software Architectures: Views and Beyond*. Boston, MA: Addison-Wesley, 2003.